VR Builder Pro

Add-on for VR Builder

Table of Contents

Introduction

This add-on contains a collection of animation behaviors that allow VR Builder to display more complex animations than what is possible with the built-in tools.

When installed, the `Move Object` core behavior will be disabled in the menu, as its functionality is 100% included in the new `Animate Transform` behavior. To manually enable it, go to `Tools > VR Builder > Developer > Allowed Menu Items Configuration`.

Quick Start

The easiest way to get started with this add-on is to check out the included demo scene.

If it is the first time you open the demo scene, you will have to do it through the menu: `Tools > VR Builder > Demo Scenes > Animations`. This is necessary as a script will copy the demo course in the StreamingAssets folder. After the first time, the demo scene can be opened normally.

Press Play to try out the behaviors included in this add-on. The demo scene includes a station for every behavior. You can teleport there and check out some practical uses of the included behaviors.

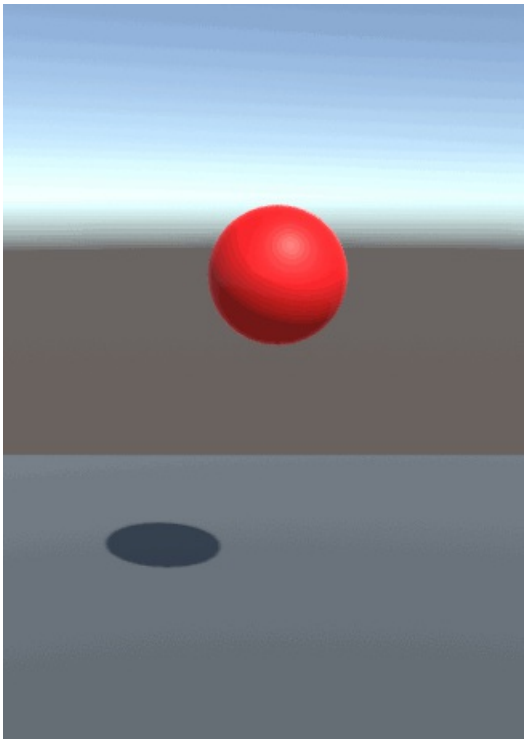
Behaviors

The Animations add-on includes the following behaviors.

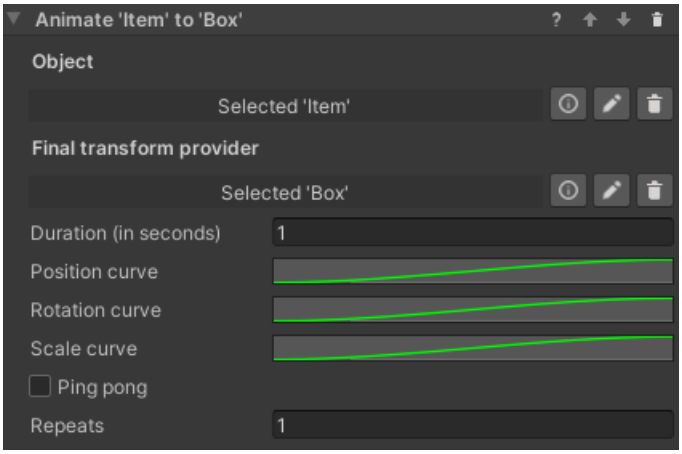
Animate Transform

Introduction

This behavior animates a game object by changing its position, rotation and scale over time until it matches those of a "transform provider" game object. It is possible to set how position, rotation and scale are animated over time through separate animation curves. The behavior can be found under `Animation > Animate Transform`.



Inspector



The **Animate Transform** behavior accepts the following parameters.

Object: The game object to be moved.

Final transform provider: The game object which provides the final position, rotation and scale of the animation.

Duration (in seconds): Duration in seconds of the animation.

Position curve, Rotation curve, Scale curve: These animation curves determine the object's transform at a given point in time. The curve can have values from 0 (the object's original position, rotation or scale) to 1 (the transform provider's position, rotation or scale). Note that the length of the curves is normalized: while it is possible to have the time axis greater or lesser than 1, this won't affect the duration of the animation - it is recommended to leave the time axis to the default length of 0 to 1.

Ping pong: If this is checked, the animation will play backwards after finishing, resulting in the object animating and then

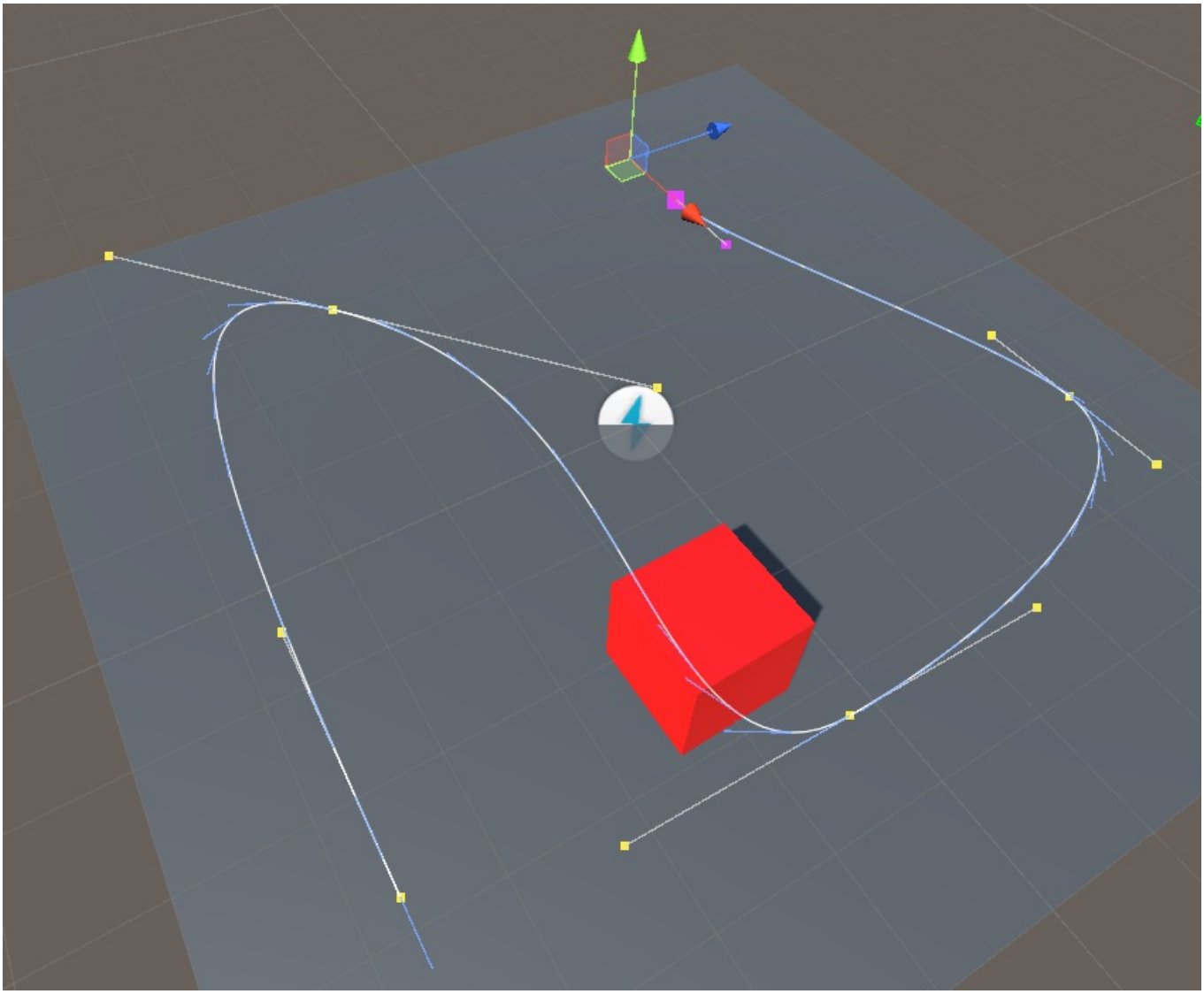
returning to the original position. Note the total duration will be twice the value in the `Duration` field. This is similar to creating a symmetrical velocity curve, like for example a bell shape.

Repeats: The number of times the animation will repeat. Note that each repeat will increase the duration of the animation by its full amount. If ping pong is set, it will be included in every repeat.

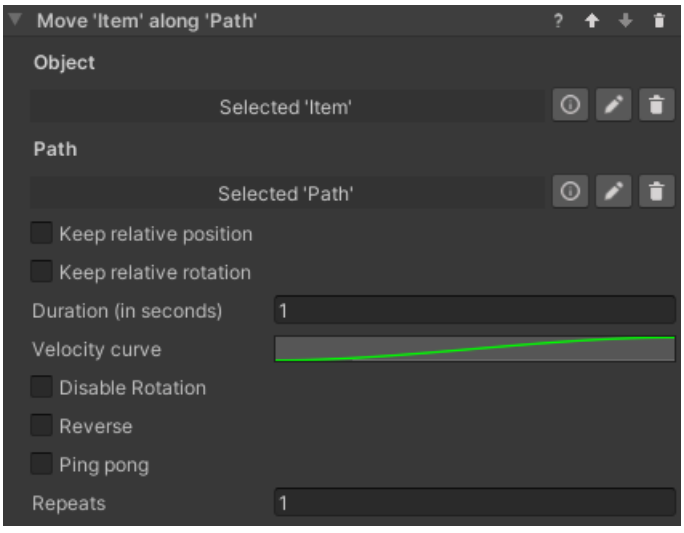
Follow Path

Introduction

This behavior animates a game object so that it follows a path, for example a spline. It is possible to set a curve determining how fast the object moves along the path, plus a number of options described below. The behavior can be found under `Animation > Follow Path`.



Inspector



The **Follow Path** behavior accepts the following parameters.

Object: The game object to be moved.

Path: The path the object will follow. This needs to be an `IPathProperty`, like the `BezierSplinePathProperty` provided in VR Builder Core.

Keep relative position: If unchecked, the object will be teleported on the path when the animation starts, and its position throughout the animation will be on the path itself. If checked, the object will retain its current position and move parallel to the path while animating.

Keep relative rotation: If unchecked, the object will rotate so that its forward vector follows the direction of the path throughout the animation. If checked, the object will retain its current orientation, but still rotate following the direction of the path.

Duration (in seconds): Duration in seconds of the animation.

Velocity curve: This animation curve determines the object's position on the path at a given point in time. The position on the path can be a value from 0 (start) to 1 (end). Note that the curve length is normalized: while it is possible to have the time axis greater or lesser than 1, this won't affect the duration of the animation - the curve will be extended or compressed to fit the provided time duration. The first key of the curve should always be at 0 on the horizontal axis.

Disable Rotation: If enabled, the object will not rotate while following the path and the settings on *Keep relative rotation* will be ignored.

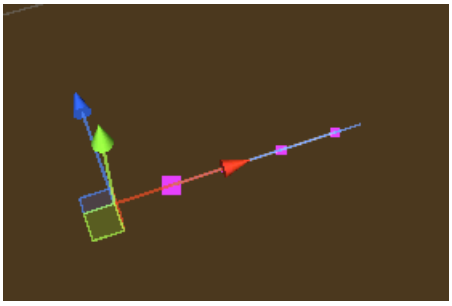
Reverse: Utility checkbox that plays the animation backwards. It is equivalent to mirroring the velocity curve.

Ping pong: If this is checked, the animation will play backwards after finishing, resulting in the object animating and then returning to the original position. Note the total duration will be twice the value in the `Duration` field. This is similar to creating a symmetrical velocity curve, like for example a bell shape.

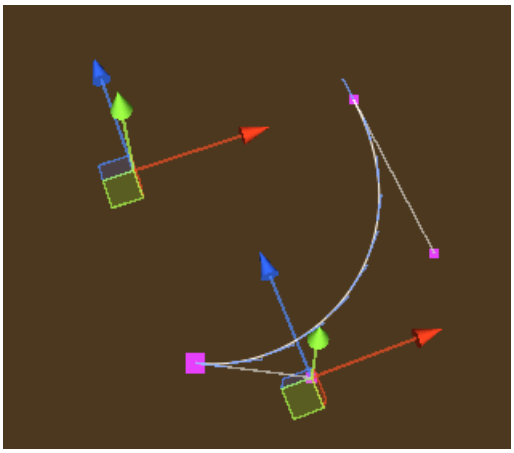
Repeats: The number of times the animation will repeat. Note that each repeat will increase the duration of the animation by its full amount.

The Bezier Spline Path Property

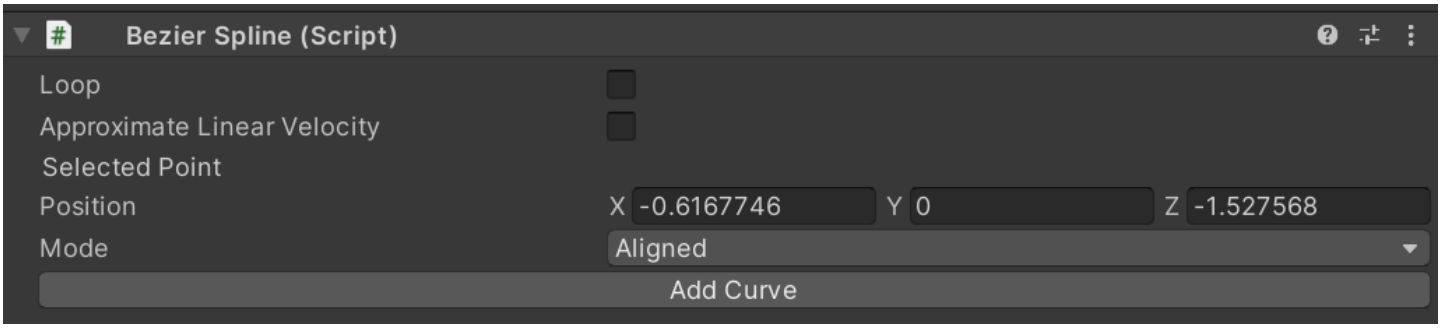
The `BezierSplinePathProperty` is an implementation of the `IPathProperty` interface included in VR Builder core, and can thus be used to create paths for the **Follow Path** behavior. It's recommended to add it to an empty game object. It will automatically add a `BezierSpline` component, which will display a default 4 point Bezier curve in the scene.



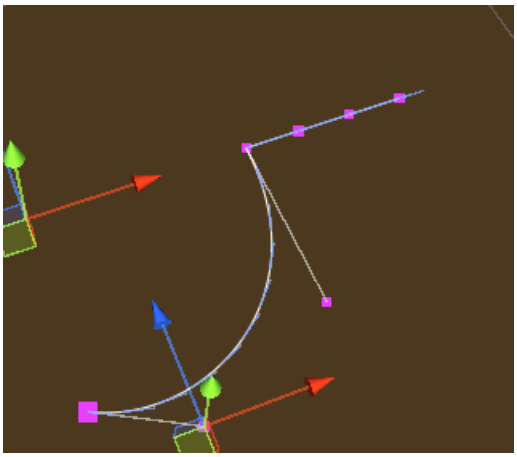
It is possible to select and move the points in 3D space to manipulate the curve.



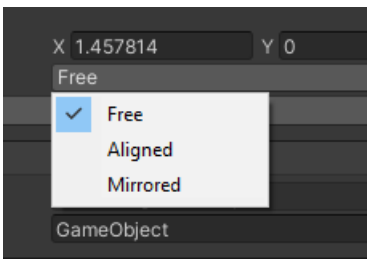
We can see the position of the currently selected point in the inspector.



By clicking **Add Curve** in the `BezierSpline` inspector, we can add a second bezier curve connected to the current one.



With a point selected in the inspector, it is possible to change the point mode.



The color of the point changes depending on the mode selected. The following modes are available.

Free (Magenta): The handles of the adjacent curves are independent, and can form a sharp angle if not aligned.

Aligned (Yellow): The handles of the adjacent curves are aligned, so there will be a smooth transition, but their length can be set individually.

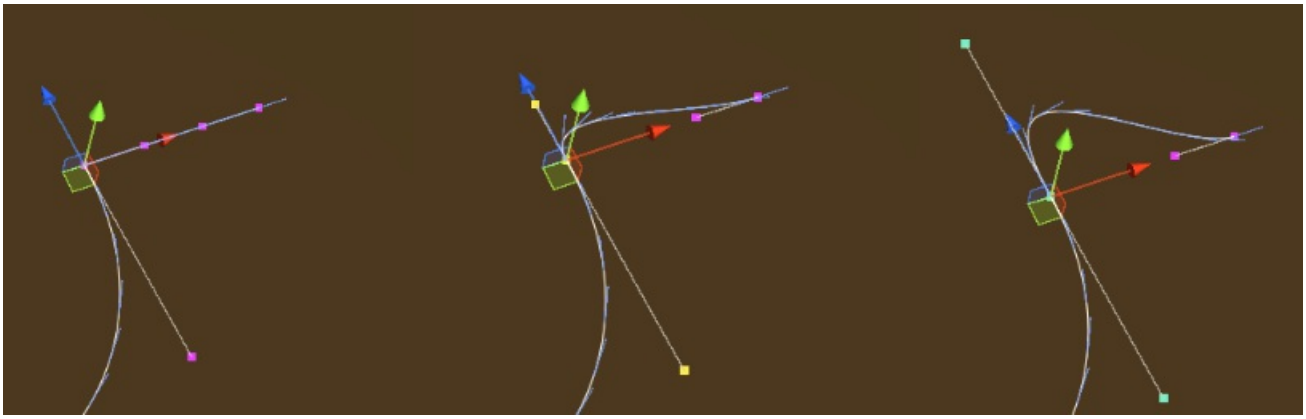
Mirrored (Cyan): The handles of the adjacent curves are aligned and of equal length.

Other options are:

Loop: Will close the path to form a loop. Especially useful with repeats, as the object will keep going around the path.

Approximate Linear Velocity: Normally, velocities on a Bezier curve are non-linear. This means that, by default, the object's speed will change depending on where it is on the path and which curve it is on. Enabling this option will make the object approximate a linear speed, which means that the animation speed will be actually more faithful to the animation curve.

Granularity of Approximation: This parameter is only exposed if `Approximate Linear Velocity` is selected. It determines the number of segments each curve will be subdivided in, a higher value will result in a more constant speed along the path, very low values (less than 10) can cause the object to change speed in a strange way. Lowering the value can increase performance.

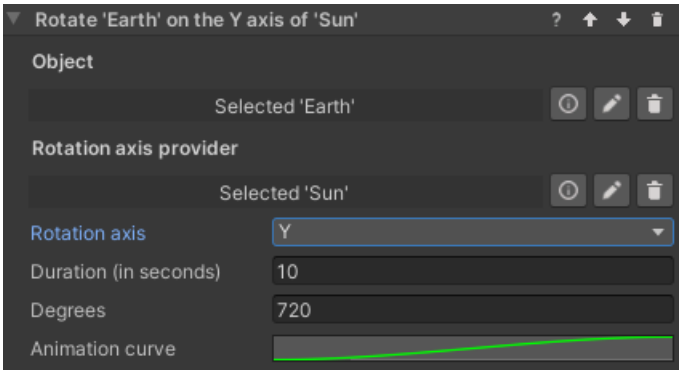


Rotate Around Axis

Introduction

This behavior rotates an object around a given axis. The object rotates a specified amount of degrees in a given time. The axis can be represented by a second transform, and it is possible to choose whether the object will rotate around the X, Y or Z axis of that object.

Inspector



It is possible to configure the following parameters.

Object: The object to be rotated.

Rotation Axis Provider: The object defining the position of the rotation axis. If none is selected, the axis will pass through the origin of the rotating object, which in most cases means that the object will rotate on itself.

Rotation Axis: The local axis of the provider object which will be used to rotate around.

Duration (in seconds): The total duration of the animation.

Animation Curve: Defines the state of the animation over time. The X axis represents the duration of the animation (normalized), while the Y axis represents values from the object's initial rotation (0) and the object's target rotation defined above (1).

Set Animator Parameter

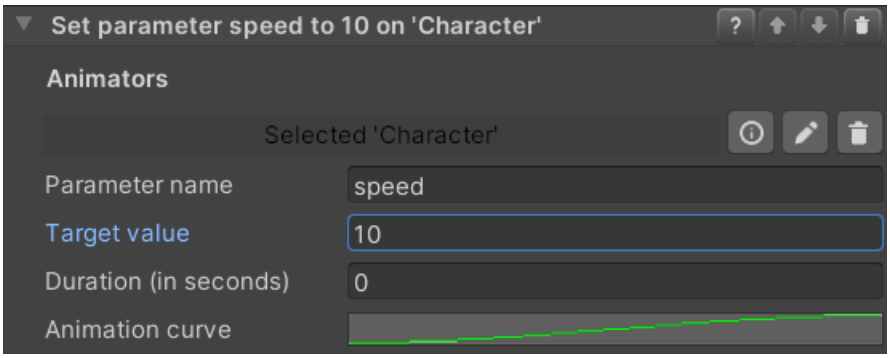
Introduction

This set of behaviors allow to control the parameters of multiple `Animator` components from within VR Builder. The included behaviors are:

- Set Animator Trigger Parameter
- Set Animator Boolean Parameter
- Set Animator Integer Parameter
- Set Animator Float Parameter

These behaviors set the specified parameter to the desired value immediately. The float variant can additionally change the parameter over time, following an animation curve.

Inspector



The inspector for these behaviors works similarly for all variants, although not all variants have all options. The behavior makes use of the following parameters.

Animator: `Process Scene Objects` containing an `Animator Property`. As usual, it is possible to automatically add the property, but the user should ensure there is a configured `Animator` component present on the same game object as the property.

Parameter name: The name of the parameters in the Animator as a string.

Target value (not present on trigger): The desired new value for the parameter. It is not present on the Set Animator Trigger behavior as triggers don't store values.

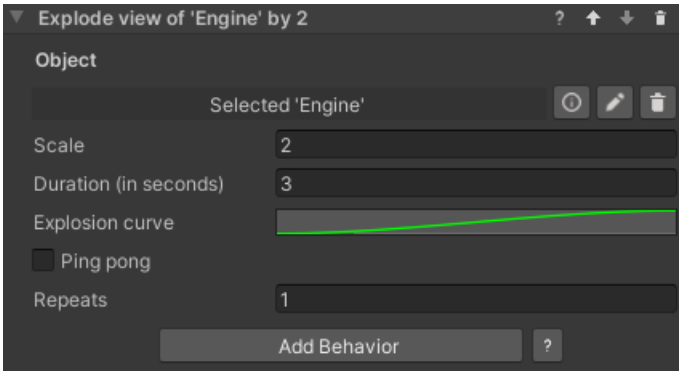
Duration (in seconds) (float only): The amount of time in seconds before the float parameter reaches its final value. If set to 0, the value will change instantly like in the other behaviors.

Animation curve (float only): Describes how the float value changes over time. The X axis represents the duration of the transition, while the Y axis represents the current value, with 0 being the initial value and 1 the target value.

Show Exploded View

Introduction

This behavior lets you easily create an exploded view of an object with a number of child objects. By default, all immediate children of the specified game objects will be displaced by the specified factor relative to their current local position. This means that children with a local position of zero will not be displaced. Note that you can create empty objects with the desired relative positions in order to achieve the desired effect.



It is also possible to manually set which child objects will be displaced. You can do so by adding them to the `Explodable Child Objects` list on the `Explodable Property` of the target object. If the list is not empty, only the objects in the list will be exploded, otherwise all first-level children will be selected.

Inspector

Object: The game object to be exploded.

Scale: How much the children will displace themselves from their initial position. A scale of 1 represent the initial position and can be used to un-explode the view. Values above 1 will explode the view, and below 1 will implode.

Duration (in seconds): Duration in seconds of the animation.

Explosion curve: This curve determines the object's position at a given point in time. The curve can have values from 0 (the child's original position) to 1 (the final position). Note that the length of the curves is normalized: while it is possible to have the time axis greater or lesser than 1, this won't affect the duration of the animation - it is recommended to leave the time axis to the default length of 0 to 1.

Ping pong: If this is checked, the animation will play backwards after finishing, resulting in the object animating and then returning to the original position. Note the total duration will be twice the value in the `Duration` field. This is similar to creating a symmetrical velocity curve, like for example a bell shape.

Repeats: The number of times the animation will repeat. Note that each repeat will increase the duration of the animation by its full amount. If ping pong is set, it will be included in every repeat.

Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review our asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.

Introduction

The Highlights and Guidance add-on provides a variety of tools to highlight important objects in the scene, direct the user towards them and otherwise show the user the path forward.

While VR Builder does include a functional Highlight Objects behavior, this add-on aims to provide more options letting you better customize your guidance and ultimately produce a better, more professional-looking application.

Quick Start

You can check out the main features of this add-on in the provided demo scene. After importing the package in a properly set-up VR Builder project, you can access the demo scene from the menu `Tools > VR Builder > Demo Scenes > Highlights and Guidance`. It is necessary to open the demo scene from the menu at least the first time, so a script will copy the required process file in the `StreamingAssets` folder.

The demo scene in this add-on is pretty simple and requires no interaction from the user. Once the scene is started, the user will find themselves in front of three cylinder shapes. These will be highlighted one by one with different highlight styles, while a voice explains what is happening. At the same time, an arrow near the feet of the user will point to the relevant cylinder.

You can press Play to try out the scene, or open the Process Editor to check out how the process is made.

Behaviors

The Highlights and Guidance add-on includes the following behaviors.

Outline Objects

Description

The Outline Objects behavior visually highlights the selected objects until the end of a step.

Configuration

- **Color**

Color in which the target object will be highlighted. Colors are defined in the RGBA or HSV color channel. By configuring the alpha (A) value, the outline can be translucent.

- **Objects**

The `Process Scene Objects` which should be highlighted.

Outline Renderer Configuration

This behaviors adds a `Outline Renderer` component to the objects to be outlined. It is possible to configure the `Outline Renderer` with further parameters, or set those parameters globally in `Project Settings > VR Builder > Component Settings`, in the `Outlines` section. Every newly created `Outline Renderer` will be configured as specified here. The parameters on the `Outline Renderer` are the following.

- **Outline Mode**

These are the different ways an object can be outlined.

- **Outline All**: the entire object is outlined.
- **Outline Visible**: only the visible part of the object is outlined.
- **Outline Hidden**: only the hidden part of the object is outlined.
- **Outline And Silouhette**: the object is outline, and the hidden part is shown as a colored silouhette.
- **Silouhette Only**: a colored silouhette is shown on the hidden part of the object.

- **Outline Color**

Specifies the color of the outline. This parameter is not relevant when using outlines with this behavior, as the behavior will override the color of the outline.

- **Outline Width**

The width of the outline. A larger value corresponds to a thicker line.

- **Precompute Outline**

This setting is not present in the global settings as it makes sense on a per-object basis. If selected, the outline is computed at editor time and stored in the object, while in the opposite case it will be generated at runtime. This might be noticeable with larger meshes. Precomputing could result in noticeably larger scene/project size, while not precomputing could cause a pause as the outline is generated on Awake().

Point at Objects

Description

The Point at Objects behavior can be used to direct the user towards the objects they are supposed to interact with. By default, this is done by displaying an arrow just in front of the user, hovering above ground level. This does not intrude the user's field of view, but they can easily look down if they need a pointer.

Configuration

- **Objects**

The `Process Scene Objects` to point at.

- **Pointer Resource**

The prefab in a `Resources` folder that is used to point at the object. The prefab needs to contain an `Object Pointer` component. By default, it points to the `CompassArrow` prefab, which displays an arrow as described above. You can select a different prefab if you wish, like for example the `CircleArrow` prefab in the `StaticAssets/Resources` folder of this add-on.

Pointer Customization

It is possible to customize the object used for pointing. The most straightforward customization is to make a copy of the `CompassArrow` prefab and replace or edit the arrow mesh.

It is also possible to create different pointer prefabs that behave in different ways. Pointer prefabs need to have a component inheriting from `ObjectPointer` on their root object. Feel free to create your own overrides with custom logic of this class. You can look at the default `ArrowObjectPointer` for reference.

Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder and this asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.

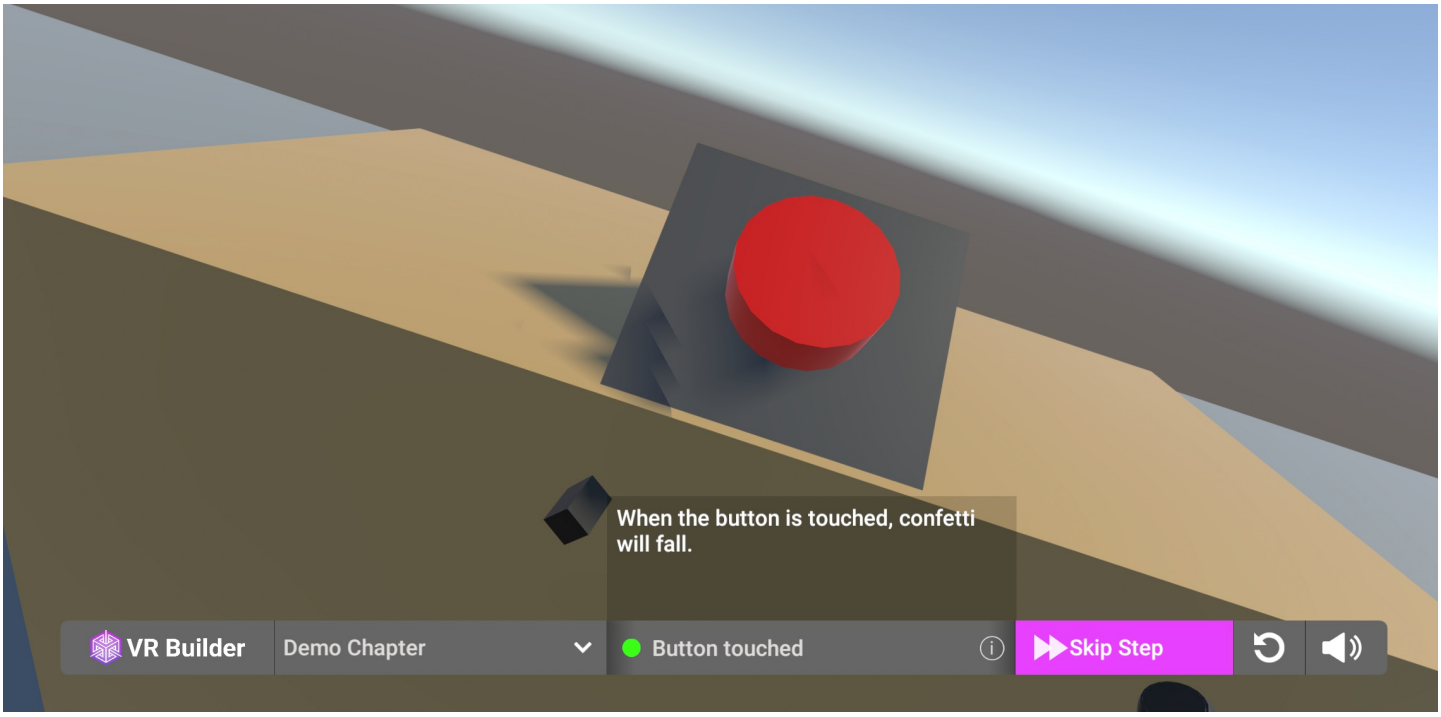
Introduction

This add-on provides examples on how to build a menu that allows to control the process execution in VR Builder. The menu allows to display the current step and its description, skip it by choosing a transition, switch chapters or restart the process.

Two prefabs are included. One displays the menu on the desktop screen, allowing an external person to control the process. This can be useful for example in training scenarios with a trainee in VR and a trainer at the computer. The other displays the menu in VR, floating in front of the user. This can be useful for standalone headsets, which lack a flat screen view, or in cases where the user is supposed to control the process themselves.

Quick Start

Drag one of the prefabs in the `Prefabs` folder in your process scene. The menu will appear when pressing Play. Feel free to try it out with the demo scene included in VR Builder!



The process starts automatically in VR Builder. If you want the process to be started from the menu, you will have to provide a custom `[PROCESS_CONTROLLER]`. You can find more information on how to do so here.

How to Use the Menu

Desktop Process Menu

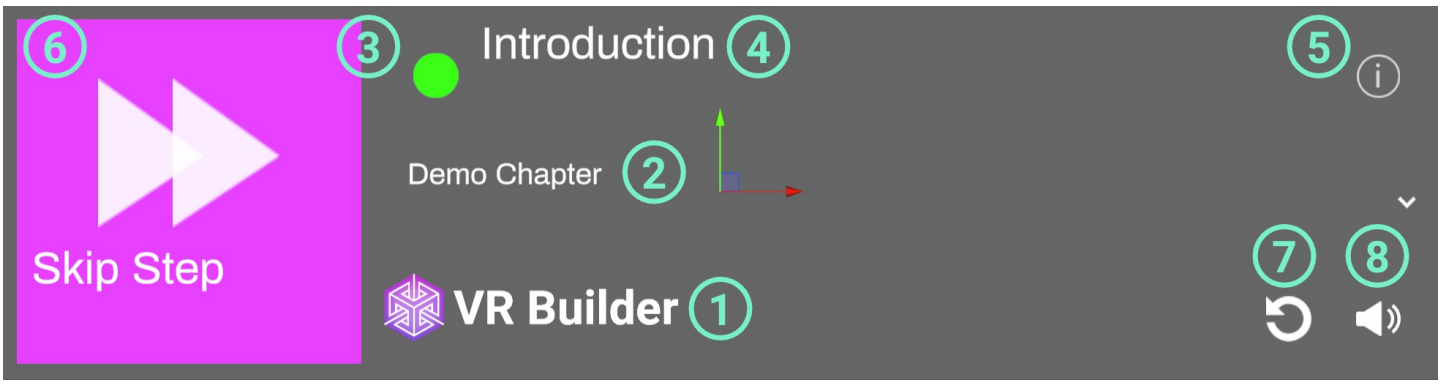
The desktop process menu is laid out as follows.

1. VR Builder logo. Feel free to replace it with your own!
2. Current chapter. You can use the drop-down to skip to a later chapter.
3. Process state indicator. It appears when the process is running.
4. Name of the current step.
5. Info button. Click it to see the description of the current step.
6. Skip Step button. Clicking this button will fast-forward to the following step. If the process is not running, it is replaced by the Start Process button.
7. Restart process button. Clicking it will restart the process from the beginning.
8. Audio toggle. Enables/disables process audio.

The desktop menu can be used by clicking with the mouse on the desired option.

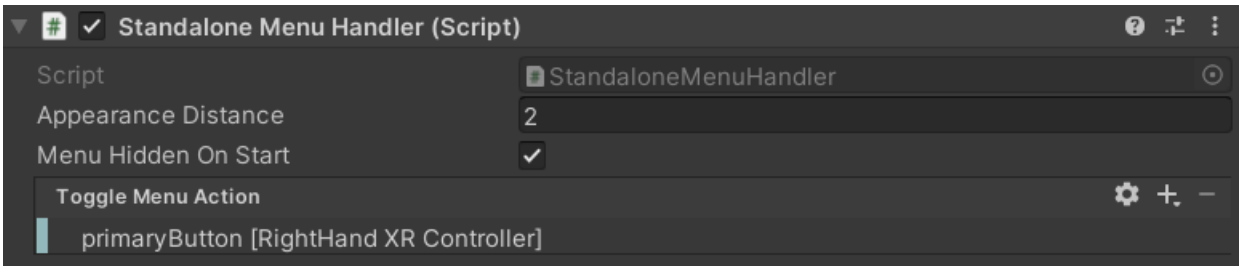
Standalone Process Menu

The standalone process menu is laid out as follows.



1. VR Builder logo. Feel free to replace it with your own!
2. Current chapter. You can use the drop-down to skip to a later chapter.
3. Process state indicator. It appears when the process is running.
4. Name of the current step.
5. Info button. Click it to see the description of the current step.
6. Skip Step button. Clicking this button will fast-forward to the following step. If the process is not running, it is replaced by the Start Process button.
7. Restart process button. Clicking it will restart the process from the beginning.
8. Audio toggle. Enables/disables process audio.

The standalone menu is designed to be opened through user input. By default, the primary button on the right controller opens the menu. This can be changed on the `Standalone Menu Handler` component on the menu prefab. In case you want the menu to be permanently present in the scene, remove the `Standalone Menu Handler` component from the prefab.

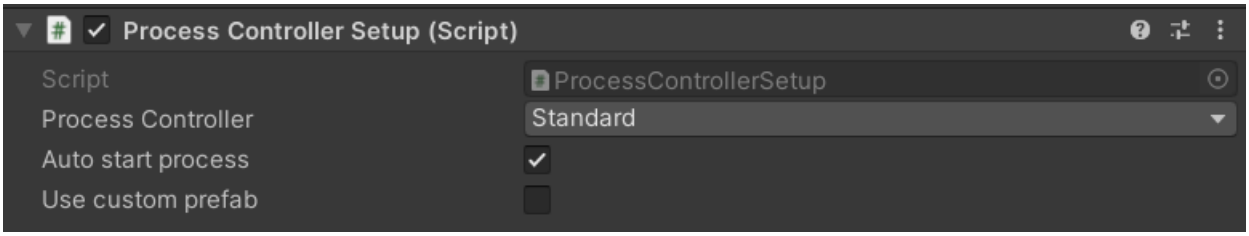


In the inspector, it is possible to select which button can be used to open and close the menu. The `Standalone Menu Handler` also takes care of repositioning the menu in front of the user every time it is opened.

To interact with the standalone menu, press the grip button on a controller to enter UI mode with that controller. This will change the hand to a pointing animation and visualize a ray originating from it. Point the ray to interactable elements on the menu and press the trigger button to activate them.

Customizing the Process Controller

The process controller can be configured on the `[PROCESS_CONTROLLER]` game object in a VR Builder scene. This object handles running the process and a few configuration parameters. The `Process Controller Setup` script lets you choose which process controller prefab is spawned when the scene runs.



It is possible to select one of the default process controller prefabs, or use a custom one by ticking the `Use custom prefab` box. It is also possible to select whether to auto start the process, or start it manually, for example through one of the menus provided in this add-on.

Both default process controllers automatically start the process when the scene runs. If you want to avoid that, so the process can be started from the menu, you'll have to create a custom process controller prefab. You can use one of the default ones found in `Assets\MindPort\VR Builder\Core\Source\Basic-UI-Component\Runtime\ProcessController\Resources\Prefabs` as a baseline.

The `Basic Process Loader` component ensures the process runs on scene start. To prevent this behavior, do not include this script in your custom controller.

Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder and this asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.

Introduction

The Randomization add-on makes it easy to add unpredictability to a VR Builder process. It includes a new type of node, called Random Branch, which allows to add random events and alternative paths with zero effort. Additionally, there are a couple tools to work with data properties: behaviors to set random booleans and numbers. You can use these to set random values for the process, and compare those values in condition, to make sure that every run of a process will be different than last time.

Quick Start

You can check out the main features of this add-on in the provided demo scene. After importing the package in a properly set-up VR Builder project, you can access the demo scene from the menu `Tools > VR Builder > Demo Scenes > Randomization`. It is necessary to open the demo scene from the menu at least the first time, so a script will copy the required process file in the `StreamingAssets` folder.

In the demo scene, the user must check and change the pressure of the tires of a car by using a provided tool. It is a freeform process, where the user can check the tires in any order and even go back to previous ones. Once done, putting the tool in the box triggers the final evaluation.

This scene uses the `Random Branch` node to randomly select a scenario when the process starts, and the `Set Random Number` behavior to set the initial tire pressure to random values.

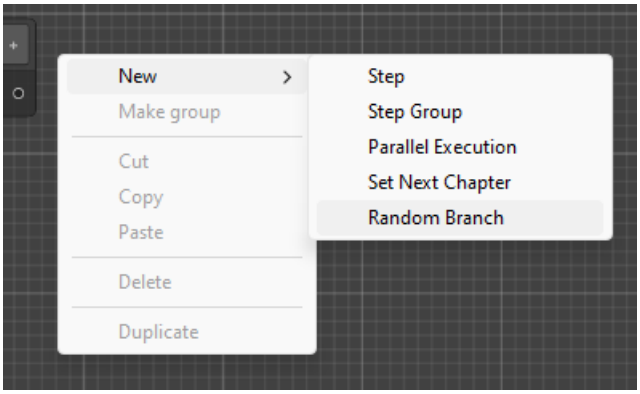
You can press Play to try out the scene, or open the Process Editor to check out how the process is made.

Additionally, you can find a tutorial on how this demo scene was created on our website.

The Random Branch Node

The Randomization add-on introduces a new type of node in addition to the standard step: the Random Branch node. This is a special kind of step that immediately routes the process to a random transition. This can be useful to add random events to your process.

To create it, select the new `Create Random Branch` option in the context menu.



The Random Branch node is designed so it can be configured directly in the Process Editor window. You can add and remove transitions at will, like any step. The fields to the left of the output port can be used to specify a weight, so it is possible for some transitions to be more probable than others.

By default, all weights are 1 and all transitions have equal chance to trigger. The weight can be any arbitrary number above or equal to zero. In the example above, the weights have a total of 5. This means that the `Very Probable Step` has a a 50% chance to be selected, while the `Improbable Step` only has one chance in ten.

Note that it can be useful to set a weight to zero for debugging purposes - such a transition will never be selected, so it is possible to steer the process through the desired nodes. If all weights in a node are equal to zero, however, the first transition will be selected.

Logging the Random Branch node

If VR Builder is set to log step output in the Project Settings, Random Branch nodes will create a log entry stating which transition has been selected.

Data Properties

This add-on makes use of data properties to store values. A data property is a VR Builder property that stores one value of a defined data type, for example a number or a boolean. It is then possible to access those in the process steps to read or change the values.

This add-on supports two types of data properties.

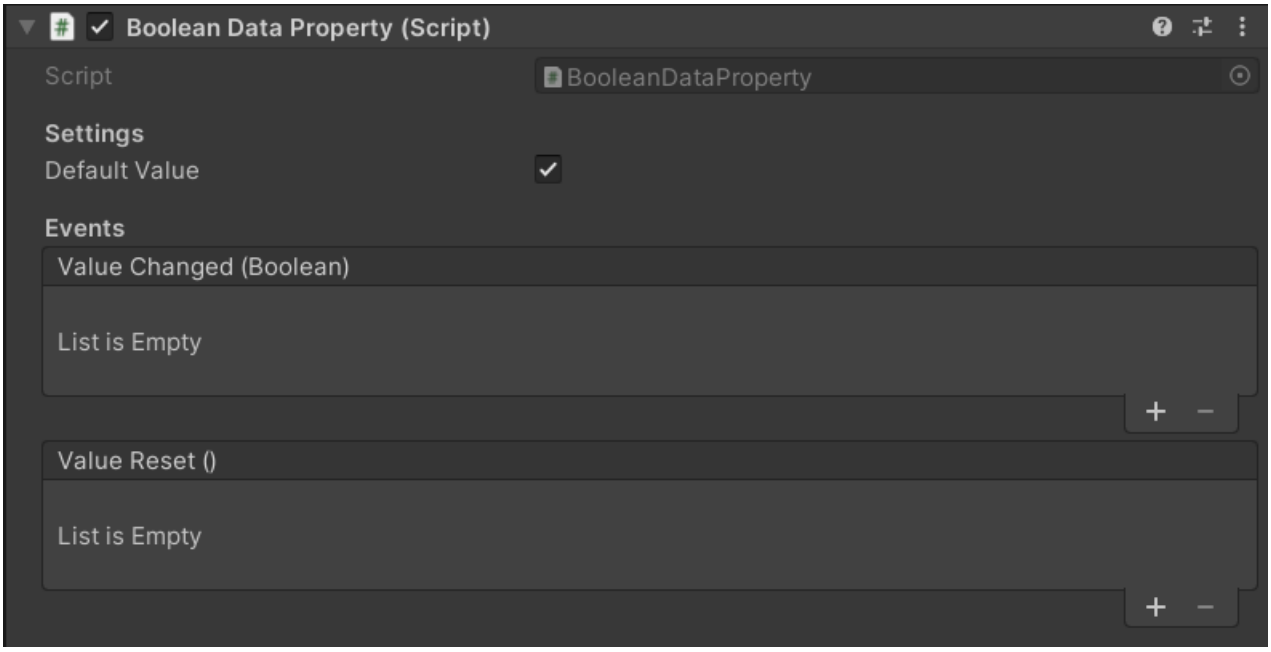
- **Number Data Property**: Stores a single number (C# type: float).
- **Boolean Data Property**: Stores a true/false value (C# type: bool).

Creating Data Properties

We consider good practice to have each data property on a different, appropriately named empty game object, e.g. "Total Points". This way it is easy to keep track of them and drag them in the step inspector when needed.

To create the property itself, just add a `Data Property` component of the required type to the game object, or do it directly in the step inspector through "Fix it" button.

In the inspector, it is possible to type a default value for the data property. The property will have that value at the start of the process, and the `Reset Value` behavior will reset the property to its default.

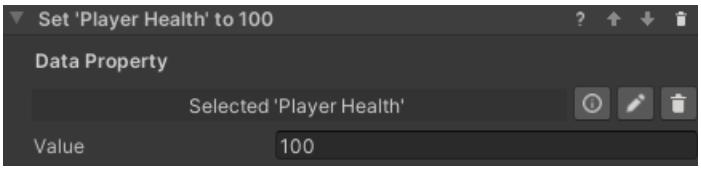


Working with Data Properties

There are some standard tools to work with data properties. These are the Set/Reset Value behaviors, which are used to change the value stored in a data property, and the Compare Values conditions, which compare two values (from data properties or constant) to check if they are fulfilled.

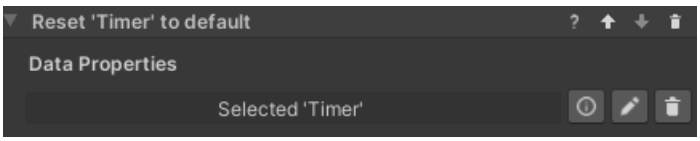
Set Value Behaviors

These behaviors set the value of a data property to a value specified in the step inspector. There is one behavior for each data property type. In the Randomization add-on, it is possible to set data properties of type Boolean and Number.



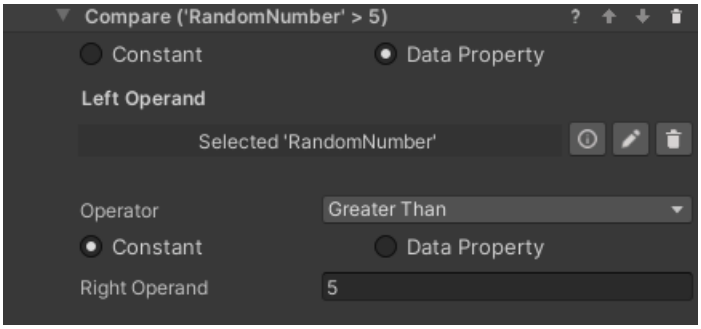
Reset Value Behavior

This behavior resets a data property's value to its default. This is zero for numerical values and false for booleans, but a different default can be specified in the inspector of the data property. The property needs to be referenced in the step inspector, and will reset when the behavior is triggered.



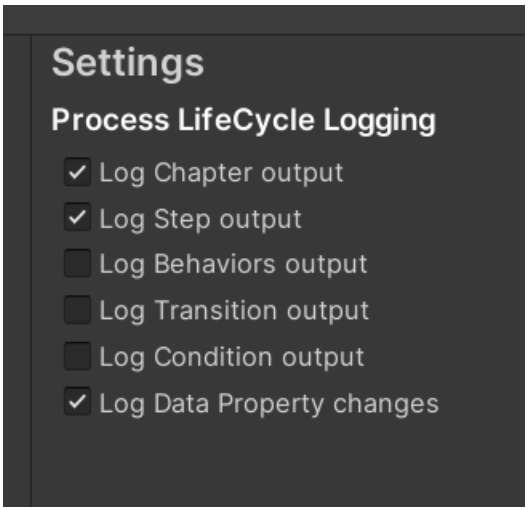
Compare Values Conditions

In the Randomization add-on it is possible to use a `Compare Numbers` or `Compare Booleans` condition. These can for example compare a random value to a constant in order to branch a process a certain way. They work in a similar way, but the comparison operators differ. You'll need to select two values and the operation between them. Use the radio buttons to select if a value comes from a data property or is a constant entered in the step inspector. In the example below, the condition will be fulfilled when the `RandomNumber` property is greater than 5.

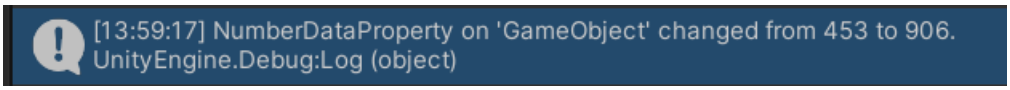


Logging Data Properties

It can be useful to log value changes to data properties in the console for debugging purposes. This can be enabled globally by ticking the relevant box in `Project Settings -> VR Builder -> Settings`.



If the `Log Data Property changes` checkbox is enabled, changes to the value of the data property will be logged in the console like the following example. Note that the name provided is the game object's name.

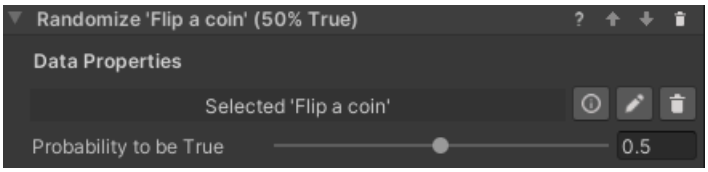


Behaviors

The behaviors included in the Randomization add-on allow you to generate random values in data properties. This section explains them in detail.

Set Random Boolean Behavior

This behavior works similarly to the *Set Boolean* behavior, except the property is not set to a specific value. Instead, it will be randomly set to *true* or *false* at runtime. It is possible to specify the probability of it to be true by moving the slider from 0 (always false) to 1 (always true).

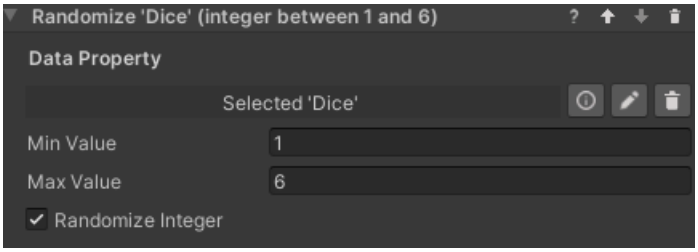


Configuration

- **Data Property**: The boolean data property to set to a random value.
- **Probability to be True**: The probability of the randomized value to be true measured from 0 (always false) to 1 (always true).

Set Random Number Behavior

This behavior sets a number data property to a random value within a range.



Configuration

- **Data Property**: The number data property to set to a random value.
- **Min Value**: The minimum value the randomized number can have.
- **Max Value**: The maximum value the randomized number can have.
- **Randomize Integer**: If checked, the randomized number will be an integer within the range. Otherwise it can be any float in the range.

Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder and this asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.

Introduction

The States and Data add-on provides developers the tools to create processes more complex and customized than before.

Thanks to the support of variables in the VR Builder process, it is possible to store values to create triggers or track performance, and to create branching processes with a variety of outcomes.

The states feature allows VR Builder process to interact with objects in the scene by changing and reading states instead of handling all the object's logic. This allows the user to quickly write custom code and easily integrate it in the VR Builder process, opening the way to endless customization and better performance.

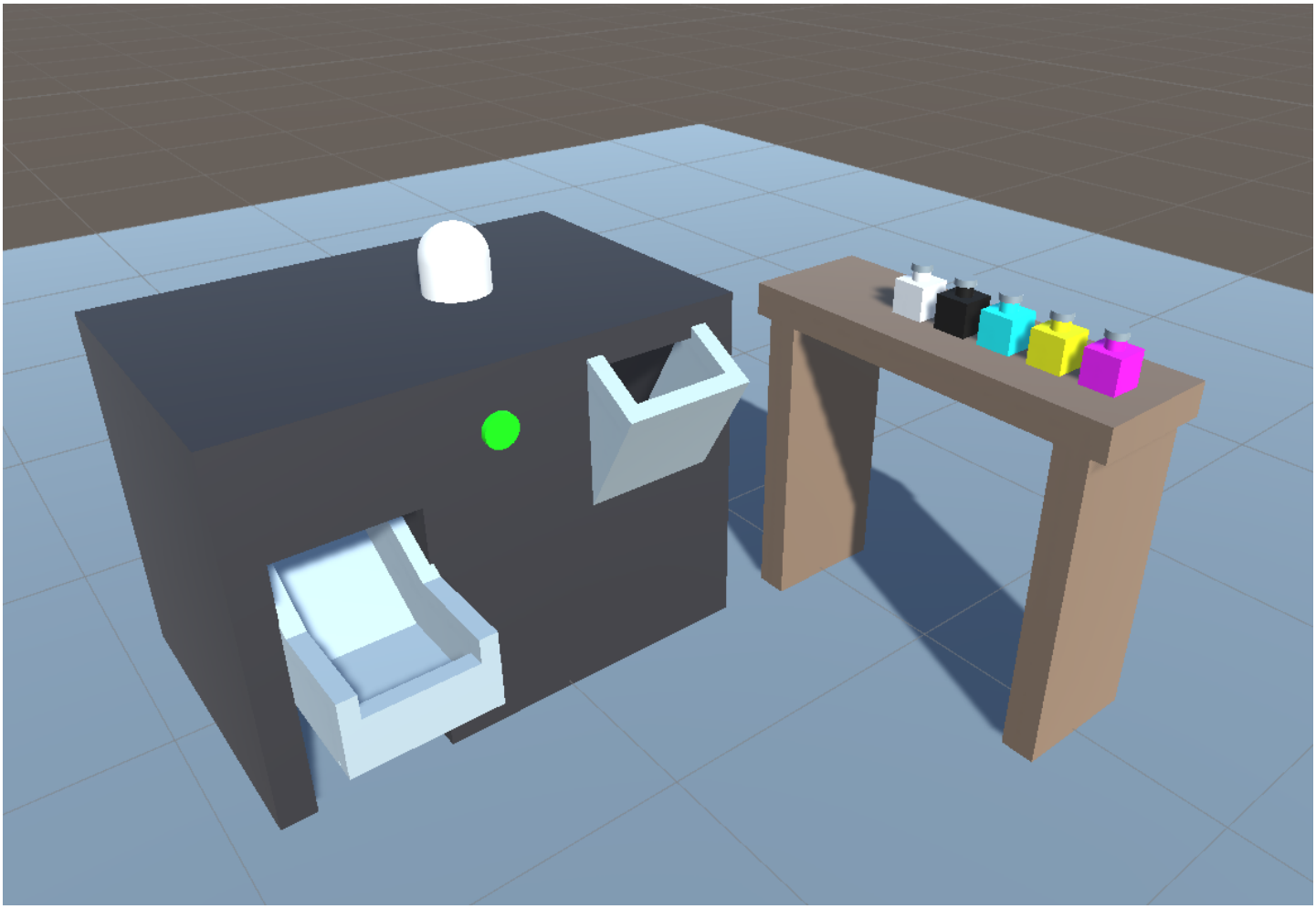
Quick Start

You can check out the main features of this add-on in the provided demo scene. After importing the package in a properly set-up VR Builder project, you can access the demo scene from the menu `Tools > VR Builder > Demo Scenes > States and Data`. It is necessary to open the demo scene from the menu at least the first time, so a script will copy the required process file in the `StreamingAssets` folder.

The demo scene consists of a color mixing machine. The user adds bottles of color in the machine to mix a new color, then can press the button to spawn a ball of the created color. It uses the data properties included in this add-on for storing and calculating the mixed color, while the machine is controlled by states.

You can press Play to try out the scene, or open the Process Editor to check out how the process is made.

Additionally, you can find a tutorial on how this demo scene was created on our website.



Data Properties

This add-on makes use of data properties to store values. A data property is a VR Builder property that stores one value of a defined data type, for example a number or a boolean. It is then possible to access those in the process steps to read or change the values.

There are four types of data properties in this add-on.

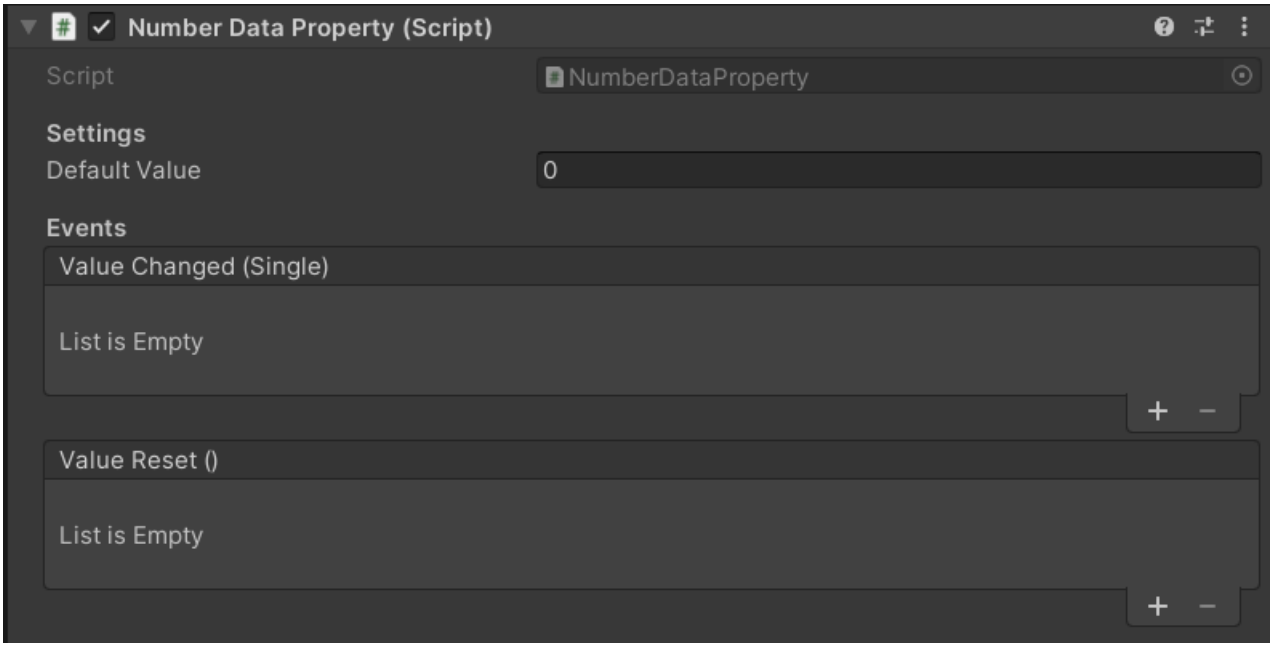
- **Number Data Property**: Stores a single number (C# type: float).
- **Text Data Property**: Stores a string of text (C# type: string).
- **Boolean Data Property**: Stores a true/false value (C# type: bool).
- **State Data Property**: Stores a state value. More detailed information below (C# type: int, exposed as enum).

Creating Data Properties

We consider good practice to have each data property on a different, appropriately named empty game object, e.g. "Total Points". This way it is easy to keep track of them and drag them in the step inspector when needed.

To create the property itself, just add a `Data Property` component of the required type to the game object, or do it directly in the step inspector through "Fix it" button.

In the inspector, it is possible to type a default value for the data property. The property will have that value at the start of the process, and the `Reset Value` behavior will reset the property to its default. Additionally, it is possible to subscribe to the property events.



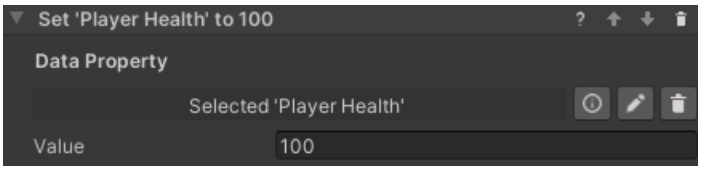
Working with Data Properties

There are some standard tools to work with data properties. These are the Set/Reset Value behaviors, which are used to change the value stored in a data property, and the Compare Values conditions, which compare two values (from data properties or constant) to check if they are fulfilled.

Set Value Behaviors

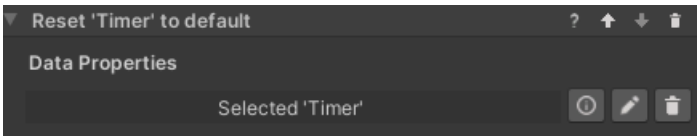
These behaviors set the value of a data property to a value specified in the step inspector. There is one behavior for each data property type. The available behaviors are listed here.

- Set Number
- Set Boolean
- Set Text
- Set State



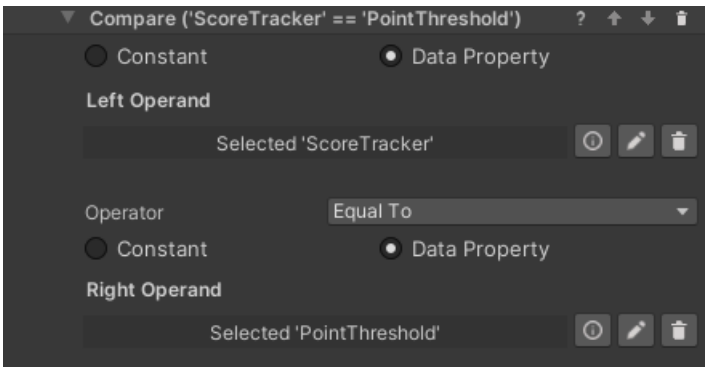
Reset Value Behavior

This behavior resets a data property's value to its default. This is zero for numerical values, false for booleans and an empty string for text, but a different default can be specified in the inspector of the data property. The property needs to be referenced in the step inspector, and will reset when the behavior is triggered.



Compare Values Conditions

In States and Data it is possible to use a `Compare Numbers`, `Compare Text` or `Compare Booleans` condition. They work in a similar way, but the comparison operators differ. You'll need to select two values and the operation between them. Use the radio buttons to select if a value comes from a data property or is a constant entered in the step inspector. In the example below, the condition will be fulfilled when the Score Tracker value is equal or greater than the Point Threshold value.



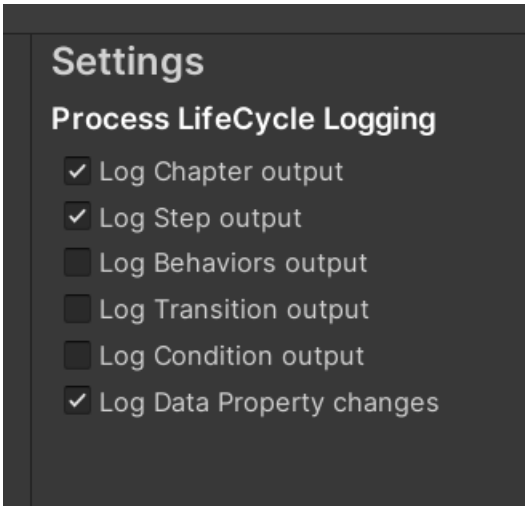
Utilities

Some data properties can have utility functions to make them handier to use with Unity events. At the moment the following utility functions are available:

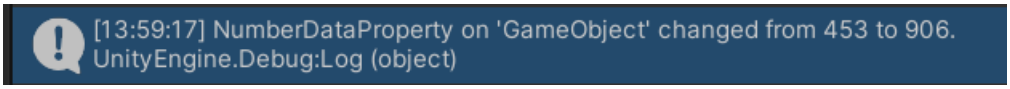
- **Number Data Property**: You can use the `IncreaseValue(float)` function to increase or decrease the value of the data property.
- **Boolean Data Property**: you can use the `InvertValue()` function to switch it from *true* to *false*, or viceversa.

Logging Data Properties

It can be useful to log value changes to data properties in the console for debugging purposes. This can be enabled globally by ticking the relevant box in `Project Settings -> VR Builder -> Settings`.



If the `Log Data Property changes` checkbox is enabled, changes to the value of the data property will be logged in the console like the following example. Note that the name provided is the game object's name.

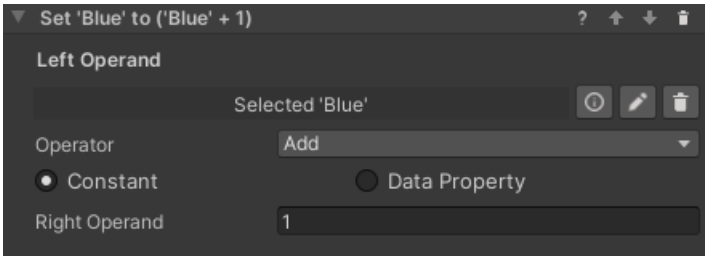


Behaviors

The behaviors included in the States and Data add-on are listed in this section.

Math Operation Behavior

The `Math Operation` behavior performs an operation on a data property and updates it with the result of the operation. It takes three parameters, `Left Operand`, `Right Operand` and `Operator`.



`Left operand` is the data property that will be changed by the operation. `Right Operand` can either be another data property or a constant value entered in the inspector. `Operator` defines the type of operation to perform. The operators currently available are listed here.

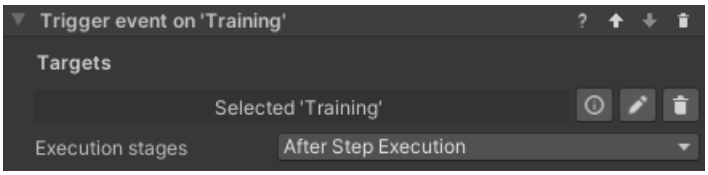
- **Add**: left + right
- **Subtract**: left - right
- **Multiply**: left * right
- **Divide**: left / right
- **Min**: change left to the lowest value between left and right
- **Max**: change left to the highest value between left and right

For example, the operation in the image will add 1 to the `Blue` data property.

Trigger Event Behaviors

This collection of behaviors let VR Builder trigger a Unity event on multiple scene objects. This gives you the freedom to bind your own functions to the event and execute custom code like it was a VR Builder behavior (although note that the behavior will end right after calling the event, so the next step could be activated immediately).

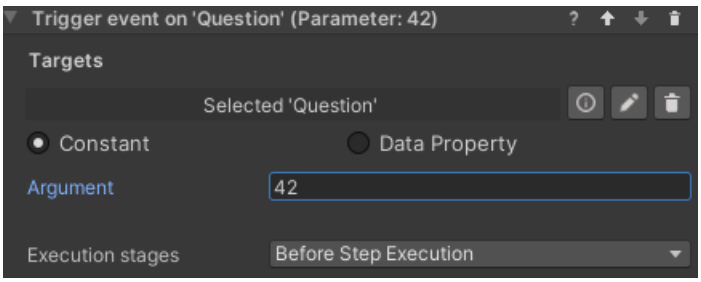
This is achieved through an event property which is added to the desired process scene object.



There are various permutations available for this behavior. It is possible to pass the following data to the event.

- No data
- Number (float)
- Text (string)
- Boolean

These value can be set in the behavior itself and can either be constant or data properties.



Trigger Event Behavior Parameters

- **Targets** References to the scene objects holding the event property we want to trigger.
- **Argument** (Behaviors with data only): Lets you specify a parameter to pass to the event. The type of the parameter (or corresponding data property) can be Number (float), Text (string) or Boolean (bool).
- **Execution stages**: Lets you select if the event is triggered as soon as the step starts, just before the step ends or both.

State Data Properties

Sometimes we need to simulate a complex behavior that is part of the environment rather than the process. For example, a machinery that can be idle, running, blocked or broken, or a door that can be locked, unlocked or open. In these case, it might make sense to code the object's logic directly with Unity Monobehaviors rather than trying to write very complex custom behaviors. This can also make sense if the object is independent from the process - for example, the machine will keep running regardless of the user's position in the process, or is designed to break randomly.

Maybe we only want VR Builder to decide when the machine starts running, or to trigger a step when it breaks, but we don't need it to be in control of what exact animation the object is into, or what sound it plays - these are characteristics that are intrinsical to the machine, so we shouldn't care about them in the process.

We can use a `State Data Property` as a communication layer between our VR Builder process and some custom logic we have written for an object. This is a data property which contains an `enum` listing all the states an object can be in. Like all data properties, it can be read and set both from the Step Inspector and from code. This allows your custom logic and the VR Builder process to interact with each other.

Creating a State Data Property

You will need to define your `enum` and create a state data property specific to it. For the machinery described above, it could look like this.

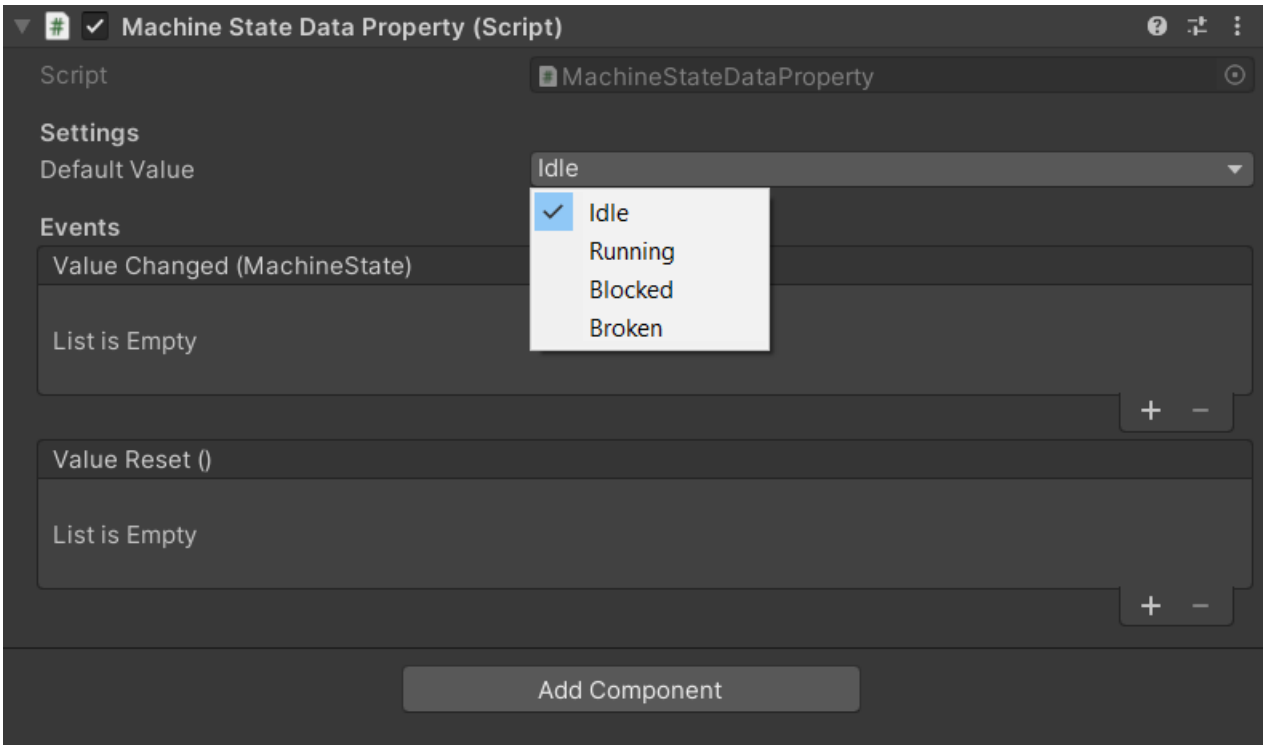
```
public enum MachineState
{
    Idle,
    Running,
    Blocked,
    Broken,
}
```

We then need to create a data property component using this enum from the generic state data property. This is just a couple lines.

```
using VRBuilder.StatesAndData.Properties;

public class MachineStateDataProperty : StateDataProperty<MachineState>
{
}
```

We'll now be able to add this as a component to a game object. As it can be seen from the drop-down for the default value, it makes use of the `MachineState` enum.



Handling States in Code

We can read and modify states from code calling the `GetState` and `SetState` methods on the data property component.

```
MachineStateDataProperty stateDataProperty = GetComponent<MachineStateDataProperty>();

// Set a state to a specified value.
stateDataProperty.SetState(MachineState.Running);

// Read the current state.
MachineState state = stateDataProperty.GetState();
```

It is also possible to subscribe to the standard data property events `ValueChanged` and `ValueReset`, respectively called when the data property value changes or is reset to its default value.

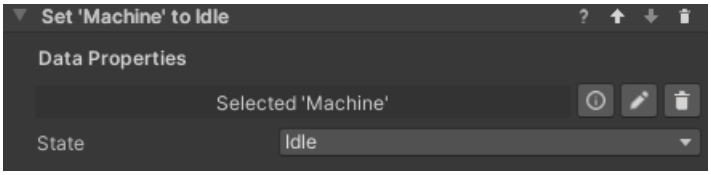
```
MachineStateDataProperty stateDataProperty = GetComponent<MachineStateDataProperty>();

// Subscribe to the value changed event.
dataProperty.ValueChanged += OnValueChanged;

// Function to be called when the event is triggered.
private void OnValueChanged(object sender, EventArgs e)
{
    // Code to be executed on value changed.
}
```

Set State Behavior

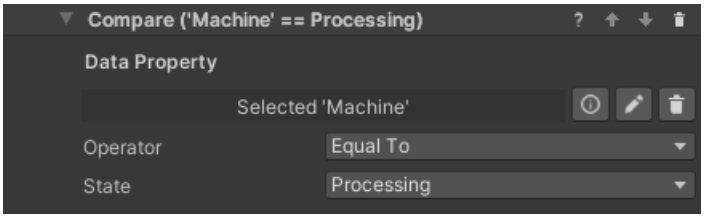
The `Set State` behavior can be used to change the property's state from the VR Builder process. It works very similarly to the `Set Value` behavior, the only difference being that the state drop-down will not be visible until a `State Data Property` is dragged in the inspector, as it depends on the provided enum.



It is also possible to use the `Reset Value` behavior on a `State Data Property`, it will work like any other data property.

Check State Condition

It is possible for a process to read a state and react accordingly using the `Check State` condition. This condition compares a `State Data Property` to a specified value, in a way similar to other conditions comparing values.



A number of operations are available. Since enum values can be ordered, it is also possible to check if the state precedes or follows the specified one.

Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder and this asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.

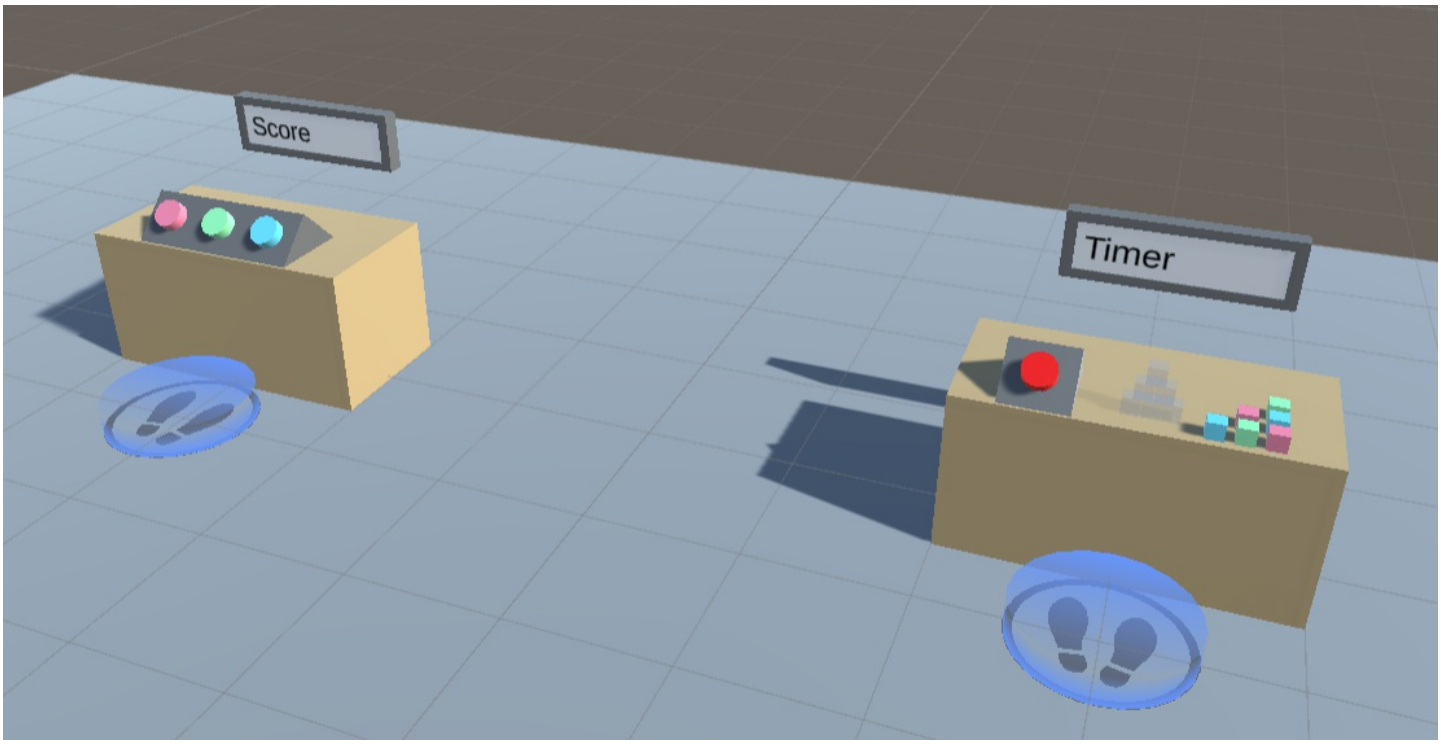
Introduction

This add-on for VR Builder focuses on tracking performance of the VR user, and provides tools like timers and point trackers. With it, it is easy to measure the time taken to perform tasks and to assign scores while providing feedback to the user.

Quick Start

You can check out the main features of this add-on in the provided demo scene. After importing the package in a properly set-up VR Builder project, you can access the demo scene from the menu `Tools > VR Builder > Demo Scenes > Track and Measure`. It is necessary to open the demo scene from the menu at least the first time, so a script will copy the required process file in the `StreamingAssets` folder.

The demo scene includes two stations, which highlight respectively point tracking and time tracking. Feel free to try those in VR. You can also open the Process Editor to see an example of how the provided conditions and behaviors can be used in a process.



Data Properties

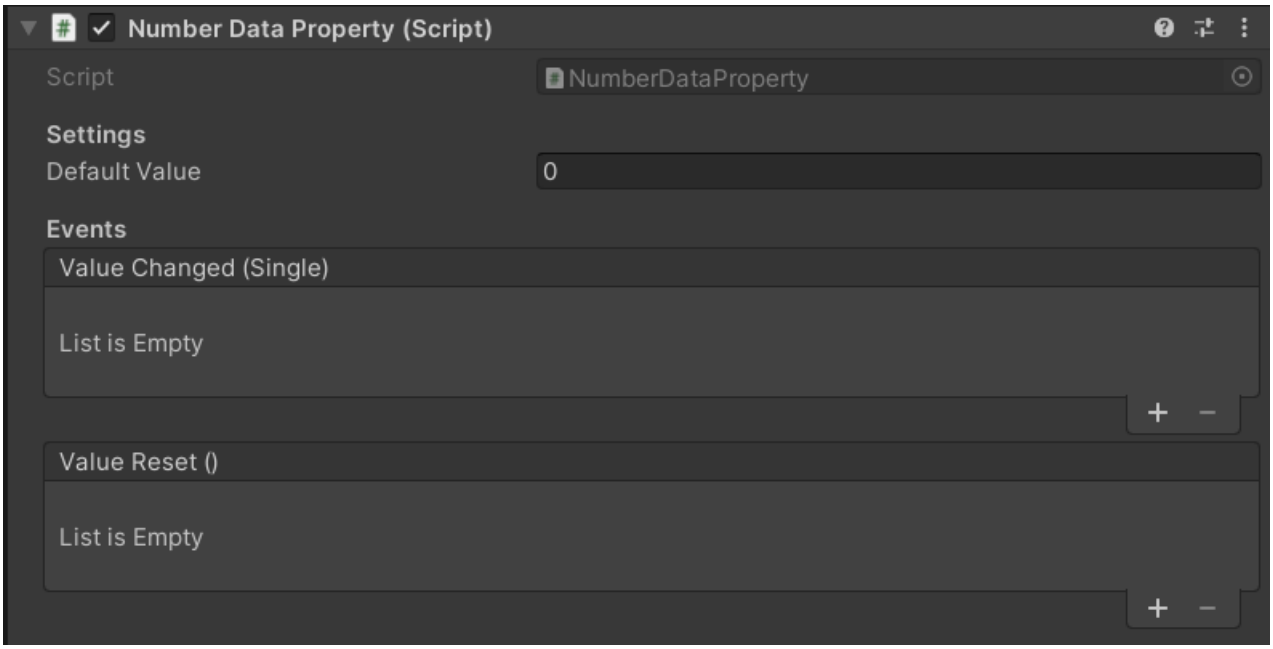
This add-on makes use of data properties to store data values. A data property is a VR Builder property that stores one value of a defined data type, for example a number or a boolean. It is then possible to access those in the process steps to read or change the values.

Creating Data Properties

We consider good practice to have each data property on a different, appropriately named empty game object, e.g. "Total Points" or "Assembly Time". This way it is easy to keep track of them and drag them in the step inspector when needed.

To create the property itself, just add a `Data Property` component of the required type to the game object, or do it directly in the step inspector through the usual "Fix it" button.

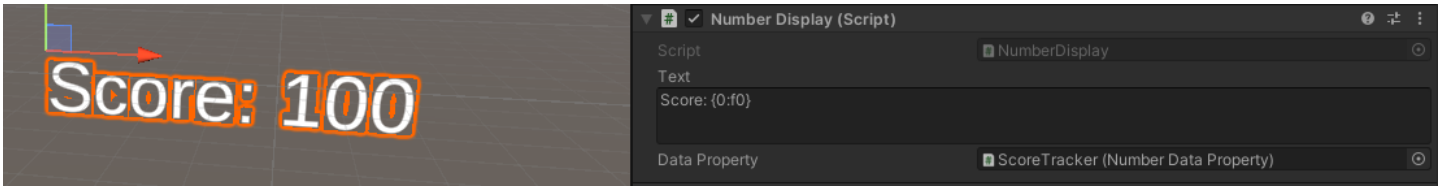
In the inspector, it is possible to type a default value for the data property. The property will have that value at the start of the process, and the `Reset Value` behavior will reset the property to that value.



Data Property Displays

While data properties are assigned to game objects, they are just abstract values, and they are not visible in the scene. Yet, it can be useful to make them visible to the user. This add-on includes a few prefabs which can visualize the contents of data properties on a text mesh. They can be used as they are, or edited and combined with your own prefabs.

Start by dragging in the scene a display of the appropriate type (number, text, boolean or time). Then reference the desired data property in the `Data Property` field of the display component on the prefab. The display should already work when the process is started.



It is possible to configure the displayed text by editing the `Text` field of the display component. The following variables can be used:

- `{0}`: The value stored in the data property.
- `{1}`: The name of the game object the property is on.

These variables support the .NET string formatting rules as detailed here. For example, you might want to limit the fractional

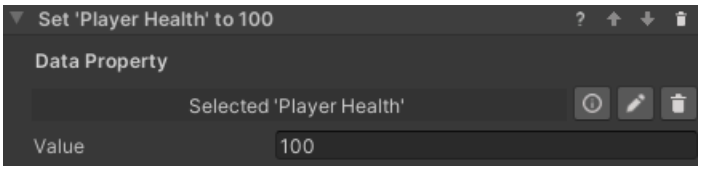
digits of a number data property. Writing the variable as `{0:f2}` will always show two fractional digits, while writing it as `{0:f0}` will display only the integer part. It is also possible to format time in the time display this way, for instance `{0:mm}:{0:ss}.{0:fff}` will display minutes, seconds and fractional digits formatted like "02:34.673".

Working with Data Properties

There are some standard tools to work with data properties. These are the Set/Reset Value behaviors, which are used to change the value stored in a data property, and the Compare Values conditions, which compare two values (from data properties or constant) to check fulfillment.

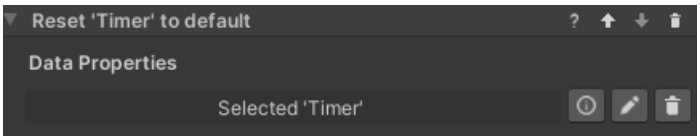
Set Value Behaviors

The Track and Measure add-on provides this behavior in two flavors: `Set Number` and `Set Boolean`. They work the same way: reference an object with a data property of the corresponding type, and input the new value. The value will change when the behavior is triggered.



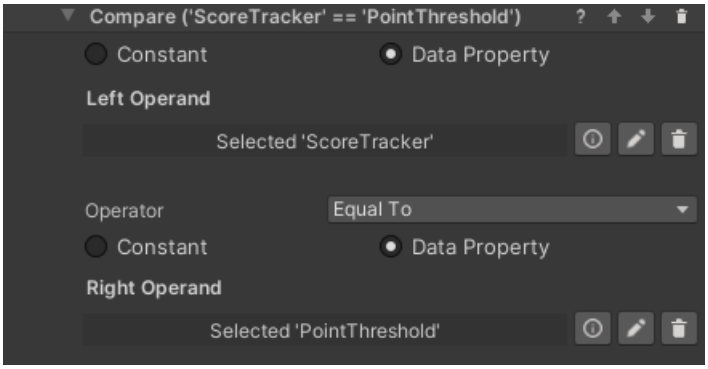
Reset Value Behavior

This behavior resets a data property's value to its default. This is zero for numerical values, false for booleans and an empty string for text, but a different default can be specified in the inspector of the data property. The property needs to be referenced in the step inspector, and will reset when the behavior is triggered. This can be useful for example for resetting a timer.



Compare Values Conditions

In Track and Measure it is possible to use a `Compare Numbers` condition or a `Compare Booleans` condition. They work in a similar way, but the comparison operators differ. You'll need to select two values and the operation between them. Use the radio buttons to select if a value comes from a data property or is a constant entered in the step inspector. In the example below, the condition will be fulfilled when the Score Tracker value is equal or greater than the Point Threshold value.

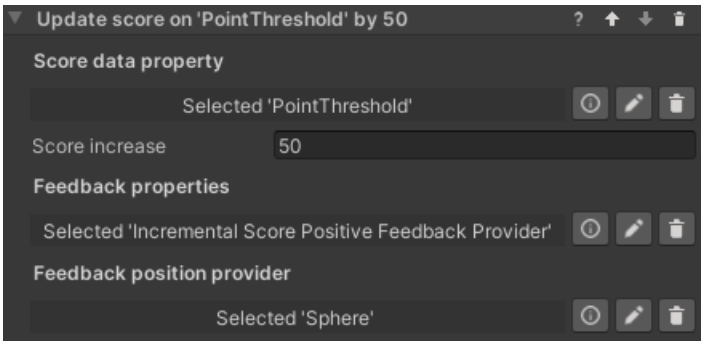


Score Tracking

One of the aims of this add-on is to provide an easy way to track scores and quantify performance in a VR process. For this, the main tool is the `Update Score` behavior, which can add or subtract from a numerical data property and trigger relevant user feedback.

Update Score Behavior

The `Update Score` behavior requires you to specify a `Number Data Property` holding the score and the amount increase. Note that this can be a negative amount. When the behavior is triggered, the value in the data property will be updated accordingly.

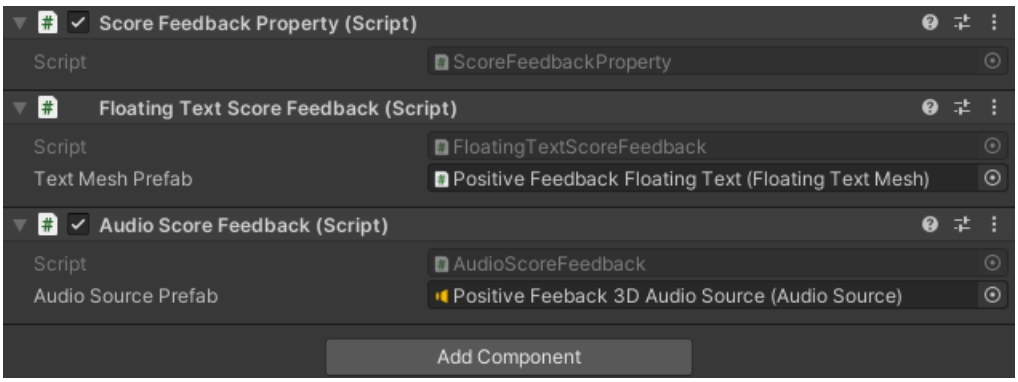


Additionally, you can add feedback to the score increase by referencing a feedback provider object in the `Feedback property` field. If you do, you can also specify a process scene object which will provide the position for position-dependent feedback, like the floating text in our default implementation.If no position provider is referenced, feedback using a position parameter will default to the feedback provider object's position.

Feedback Providers

You can customize the user feedback for a score increase by adding score feedback components to a game object with a `Score Feedback Property`. When the `Update Score` behavior triggers, every component on the game object will trigger its feedback.

For example, we offer two prefabs, one for providing positive feedback, one for negative. Both play a sound and display a floating number showing the score increase when triggered. If a position provider is specified in the behavior, both the floating number and the sound will trigger at the specified position, else it will trigger at the feedback object's position.



This happens because the game object includes both a `Floating Text Score Feedback` component and an `Audio Score Feedback` component. These can be further configured by changing the prefabs they spawn, and new components can be added.

Creating Custom Score Feedback Providers

Creating your own score feedback provider components requires minimal coding skills. Just create a component implementing the `IScoreFeedbackProvider` interface and trigger your logic in the `TriggerFeedback` method.

```
public class MyScoreFeedbackProvider : MonoBehaviour, IScoreFeedbackProvider
{
    public void TriggerFeedback(float scoreDelta, float finalScore, Transform positionProvider)
    {
        // TODO Implement feedback logic.
    }
}
```

To customize your feedback, you can use the following parameters.

- `scoreDelta`: The score increase.
- `finalScore`: The total score after the increase.
- `positionProvider`: The transform where to trigger the feedback.

Time Tracking

This add-on allows creating and managing timers which can be used for tracking performance or changing the state of the process. This is done mostly through the `Start Timer` and `Stop Timer` behaviors. Those interact with a `Timer Property` on a game object, which in turn stores the elapsed time (in seconds) in a `Number Data Property`. Since the time is stored in a standard property, it is possible to use it as any numerical data, for example by dragging the timer game object in a `Compare Values` condition, and to display it in the scene like other data properties.

Start Timer Behavior

This behavior tells a `Timer Property` to start counting time in the attached `Number Data Property`, adding to the value already stored there. If `Is countdown` is selected, the timer will count down instead, and stop when zero is reached.

Stop Timer Behavior

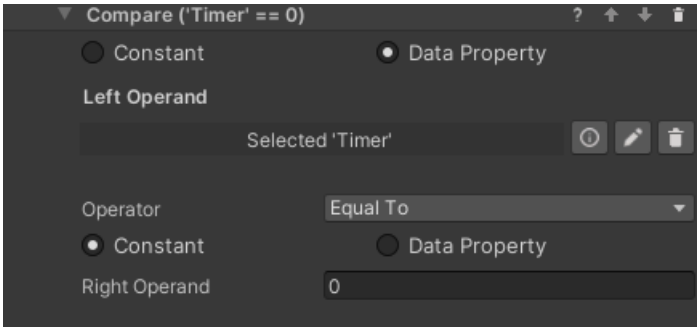
This behavior stops a running timer. It does nothing on a `Timer Property` that is not running. The `Number Data Property` will store the time at which the timer was stopped, and if the timer starts again it will start counting from that value.

Resetting a Timer

Since a timer stores its data in a `Number Data Property`, a timer can be reset to 0 (or whatever default) by executing the `Reset Value` (or the `Set Number`) behavior on the timer's game object.

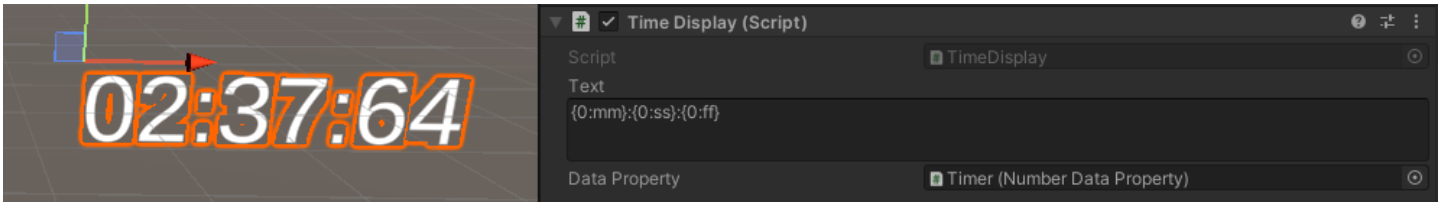
Timer at Zero Condition

It may be common to need a condition that completes when a timer reaches zero. Since the time is stored in a `Number Data Property`, no specific tool is needed - simply use a `Compare Numbers Condition` to check if the time is equal to zero. In fact, you can compare the stored time to any value, keeping in mind that the time is stored in seconds.



Displaying Time

Timers store their value in seconds in a `Number Data Property`. This means that of course a `Number Display` prefab will show that value. There is however one more prefab created specifically to show time: the `Time Display` shares many similarities with the `Number Display` but treats the value as time. By default it displays time in the mm:ss format, but that can be changed by editing the `Text` field. Since the field uses .NET formatting rules, it is possible to customize the time format as detailed here.



Contact

Join our official Discord server for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder and this asset if you like it. It will help us immensely.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.